Book Shop Management System

Course Code and Title: CO2115 – Object Oriented Programming

Submitted By:

Name	Registration Number
MM. Akeef Ahmed	EUSL/TC/IS/2022/COM/50
NF. Leena	EUSL/TC/IS/2022/COM/51
AM. Mohammed Safran	EUSL/TC/IS/2022/COM/52
ARF. Haanim	EUSL/TC/IS/2022/COM/54
KMSL. Kalupahana	EUSL/TC/IS/2022/COM/55

Submission Date: 2026-02-26

DECLARATION

We hereby declare that this assignment is entirely the result of our own group work and effort. All information, analysis, and conclusions presented are original and prepared by us. Proper references have been acknowledged where necessary. This assignment has not been submitted previously for any academic purpose at any institution.

Group Members:

Name	Signature	Date
MM. Akeef Ahmed	……	2026-02-26
NF. Leena	……	2026-02-26
AM. Mohammed Safran	……	2026-02-26
ARF. Haanim	……	2026-02-26
KMSL. Kalupahana	……	2026-02-26

TABLE OF CONTENTS

Chapter 1: Introduction

Chapter 2: Task 1 - System Development

- 2.1 System Overview
- 2.2 Objectives of the System
- 2.3 Main Functionalities
- 2.4 System Design
- 2.5 Application of OOP Concepts
- 2.6 Source Code
- 2.7 Testing and Output

Chapter 3: Task 2 - User Manual

Chapter 4: Conclusion

Chapter 5: References

Chapter 1: Introduction

The City Bookshop Management System is a Java-based desktop application developed to automate the transaction process of a bookshop. The system is designed using Object-Oriented Programming (OOP) principles and provides a graphical user interface using Java Swing.

The application supports two types of users: -

Cashier: Can view and search for book details –

Manager: Can perform all cashier functions plus add books, categories, and create user accounts

The system stores all data in text files, ensuring data persistence between sessions.

Key Technologies Used:

- Java Programming Language
- Java Swing (GUI Framework)
- File I/O for data storage

Chapter 2: Task 1 - System Development

2.1 System Overview

The City Bookshop Management System is a desktop application that helps manage bookshop operations. It provides: - User authentication (login system) - Book management (view, search, add) - Category management - User account management

Architecture:

Presentation Layer (UI) LoginFrame.java, AdminDashboardFrame.java BookAvatar.java, CashierDashBoard.java, RoundedButton.java
Service Layer DataStore.java
Model Layer Book, Category, User, Admin, Cashier

2.2 Objectives of the System

1. To automate the book management process of City Bookshop
2. To provide different access levels for Cashiers and Managers
3. To enable easy searching of books by title, category, or price
4. To maintain persistent data storage using text files
5. To demonstrate Object-Oriented Programming concepts in a real-world application

2.3 Main Functionalities

Cashier Functionalities:

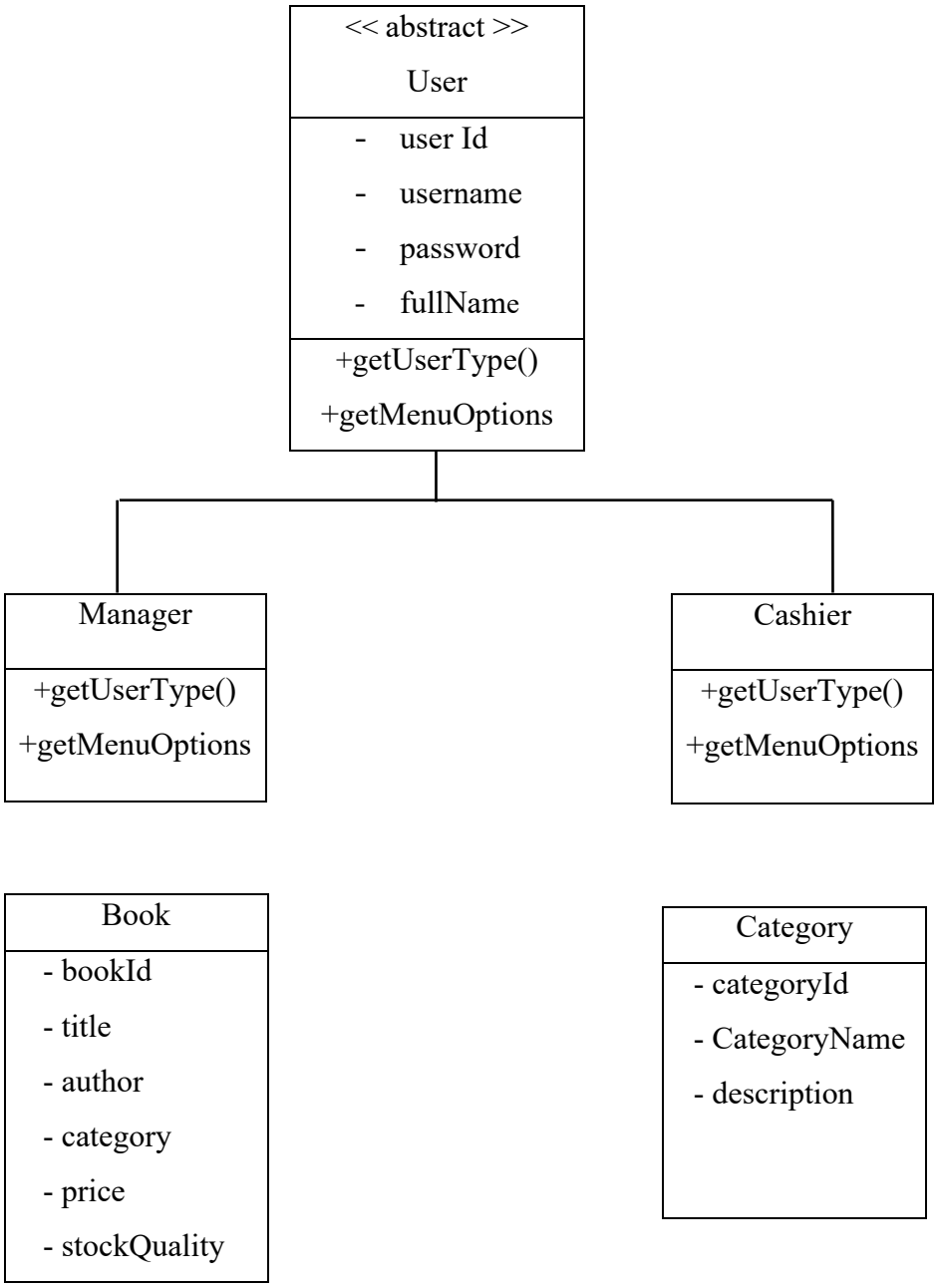
Feature	Description
View All Books	Display all books in a table format
Search by Title	Find books by their title
Search by Category	Find books by category name
Search by Price	Find books within a price range

Manager Functionalities (includes all Cashier features):

Feature	Description
Add New Book	Add a new book with details
Add New Category	Create new book categories
Create Account	Create new Cashier or Manager accounts

2.4 System Design

Class Diagram:



2.5 Application of Object-Oriented Concepts

2.5.1 Object

An object is an instance of a class. In this system, objects are created from classes to represent real-world entities.

Example from Project:

```
// Creating a Book object
Book newBook = new Book("B001", "Java Programming", "James Gosling",
"Programming", 45.99, 25);

// Creating a Manager object
Manager admin = new Manager("U001", "admin", "admin123", "System Admin");

// Creating a Cashier object
Cashier cashier = new Cashier("U002", "cashier", "cash123", "John Doe");
```

2.5.2 Class

A class is a blueprint or template for creating objects. It defines the properties (attributes) and behaviors (methods) that objects of that class will have.

Example from Project:

Book Class - Blueprint for book objects:

```
public class Book implements Serializable {
    // Attributes (properties)
    private String bookId;
    private String title;
    private String author;
    private String category;
    private double price;
    private int stockQuantity;

    // Constructor
```

```java
    public Book(String bookId, String title, String author, String category, double
price, int stockQuantity) {
        this.bookId = bookId;
        this.title = title;
        // ...
    }

    // Methods (behaviors)
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
}
```

2.5.3 Abstraction

Abstraction is the concept of hiding complex implementation details and showing only the necessary features. In Java, this is achieved using abstract classes and interfaces.

Example from Project:

The User class is abstract because:

1. We never create a generic "User" - users are always Managers or Cashiers
2. It defines abstract methods that must be implemented by subclasses

```java
public abstract class User {
    protected String userId;
    protected String username;
    protected String password;
    protected String fullName;

    // Abstract methods - no implementation, must be defined by subclasses
    public abstract String getUserType();
    public abstract String getMenuOptions();
}
```

How Abstraction is Used: - The User class hides the complexity of different user types - Child classes (Manager, Cashier) provide specific implementations - Other parts of the code work with User type without knowing the specific type

2.5.4 Encapsulation

Encapsulation is the bundling of data (attributes) and methods that operate on that data within a single unit (class), while restricting direct access to some components. This is achieved using: - **Private** fields (data hiding) - **Public** getter and setter methods (controlled access)

Example from Project:

```java
public class Book implements Serializable {
    // PRIVATE fields - cannot be accessed directly from outside
    private String bookId;
    private String title;
    private double price;

    // PUBLIC getter - controlled way to READ the value
    public String getBookId() {
        return bookId;
    }

    // PUBLIC setter - controlled way to MODIFY the value
    public void setBookId(String bookId) {
        this.bookId = bookId;
    }

    public double getPrice() {
        return price;
    }
```

```java
    public void setPrice(double price) {
        // Can add validation here, e.g., price must be positive
        this.price = price;
    }
}
```

Benefits Demonstrated:

1. Data is protected from unauthorized access
2. Validation can be added in setters
3. Internal implementation can change without affecting other code

2.5.5 Inheritance

Inheritance is a mechanism where a new class (child/subclass) inherits properties and methods from an existing class (parent/superclass). This promotes code reuse.

Example from Project:

```java
// Parent class (Superclass)
public abstract class User {
    protected String userId;
    protected String username;
    protected String password;
    protected String fullName;

    public User(String userId, String username, String password, String fullName) {
        this.userId = userId;
        this.username = username;
        this.password = password;
        this.fullName = fullName;
    }
}

// Child class - Manager EXTENDS User
```

```java
public class Manager extends User {
    public Manager(String userId, String username, String password, String fullName)
{
        super(userId, username, password, fullName);  // Call parent constructor
    }
}

// Child class - Cashier EXTENDS User
public class Cashier extends User {
    public Cashier(String userId, String username, String password, String fullName)
{
        super(userId, username, password, fullName);  // Call parent constructor
    }
}
```

Inheritance Hierarchy:

```
        User (Parent)
          /   \
Manager (Child)   Cashier(Child)
```

Keywords Used: - extends - Used to inherit from a class - super() - Used to call parent class constructor

2.5.6 Polymorphism

Polymorphism means "many forms". It allows objects of different classes to be treated as objects of a common parent class, with each responding differently to the same method call.

Example from Project:

Method Overriding:

```java
// In Manager class
@Override
```

```java
public String getUserType() {
    return "Manager"; // Returns "Manager"
}

@Override
public String getMenuOptions() {
    return "1. View All Books\n2. Search Books\n3. Add New Book\n4. Add New
Category\n5. Create New Account\n6. Logout";
}

// In Cashier class
@Override
public String getUserType() {
    return "Cashier";  // Returns "Cashier"
}

@Override
public String getMenuOptions() {
    return "1. View All Books\n2. Search Books\n3. Logout";
}
```

Polymorphism in Action:

```java
// Both are User type, but behave differently
User user1 = new Manager("U001", "admin", "pass", "Admin");
User user2 = new Cashier("U002", "cashier", "pass", "Cashier");

System.out.println(user1.getUserType());  // Output: "Manager"
System.out.println(user2.getUserType());  // Output: "Cashier"
```

Using instanceof in DashboardFrame:

```java
// Show different menu options based on user type
if (currentUser instanceof Manager) {
    menuPanel.add(addBookBtn);      // Manager only
```

```
    menuPanel.add(addCategoryBtn);  // Manager only
    menuPanel.add(createAccountBtn);// Manager only
}
```

2.6 Source Code

Project Structure:

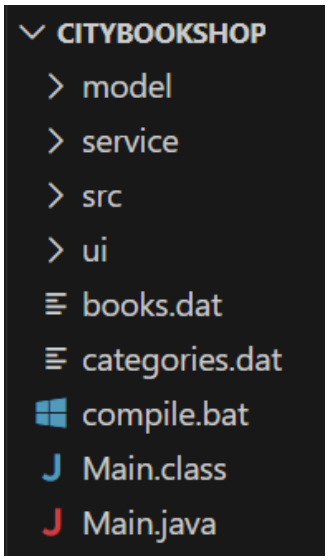


Figure 2.6.1

File 1: Main.java

Location: BookShop/Main.java

Purpose: Entry point of the application

File 2: model/Book.java

Location: BookShop/model/Book.java

Purpose: Book entity class demonstrating Encapsulation

File 3: model/Category.java

Location: BookShop/model/Category.java

Purpose: Category entity class

File 4: model/User.java

Location: BookShop/model/User.java

Purpose: Abstract class demonstrating Abstraction

File 5: model/Manager.java

Location: BookShop/model/Manager.java

Purpose: Manager class demonstrating Inheritance and Polymorphism

File 6: model/Cashier.java

Location: BookShop/model/Cashier.java

Purpose: Cashier class demonstrating Inheritance and Polymorphism

File 7: service/FileHandler.java

Location: BookShop/service/FileHandler.java

Purpose: Handles file operations for data persistence

File 8: ui/LoginFrame.java

Location: BookShop/ui/LoginFrame.java

Purpose: Login window GUI

File 9: ui/DashboardFrame.java

Location: BookShop/ui/AddnimDashboardFrame.java

Purpose: Main dashboard GUI

2.7 Testing and Output

The system was tested with the following test cases:

Test Case 1: Manager Login

- **Input:** Username: admin, Password: admin123
- **Expected Output:** Login successful, Dashboard with all options visible
- **Result:** PASS

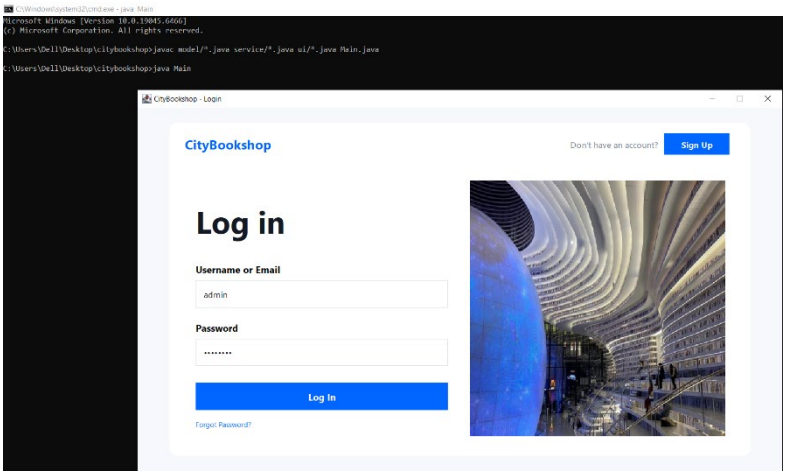


Figure 2.7.1

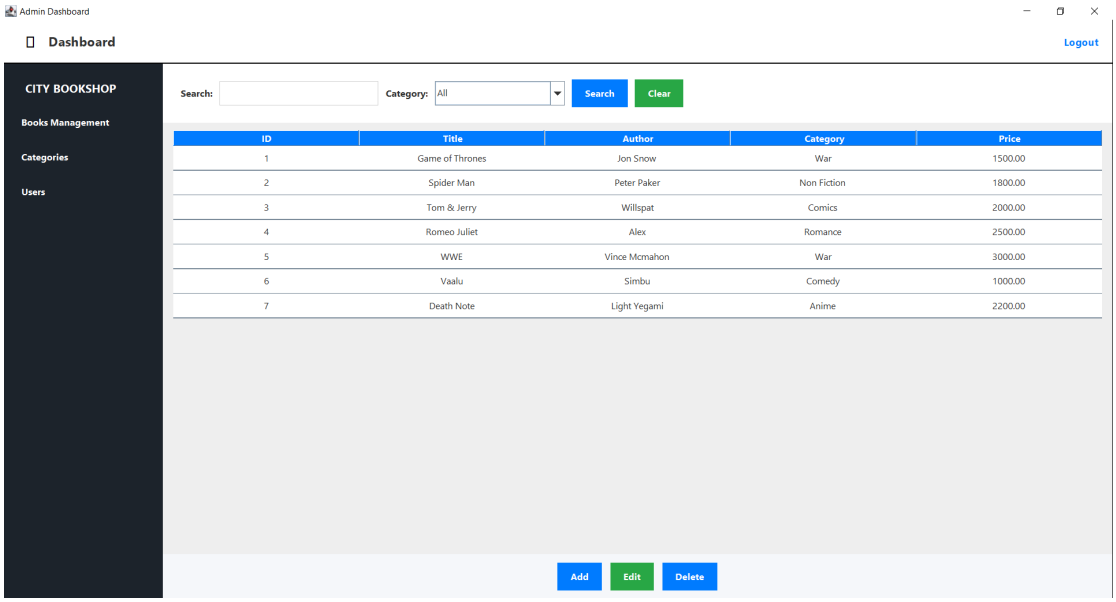


Figure 2.7.2

Test Case 2: Cashier Login

- **Input:** Username: cashier, Password: cash123
- **Expected Output:** Login successful, Dashboard with limited options
- **Result:** PASS

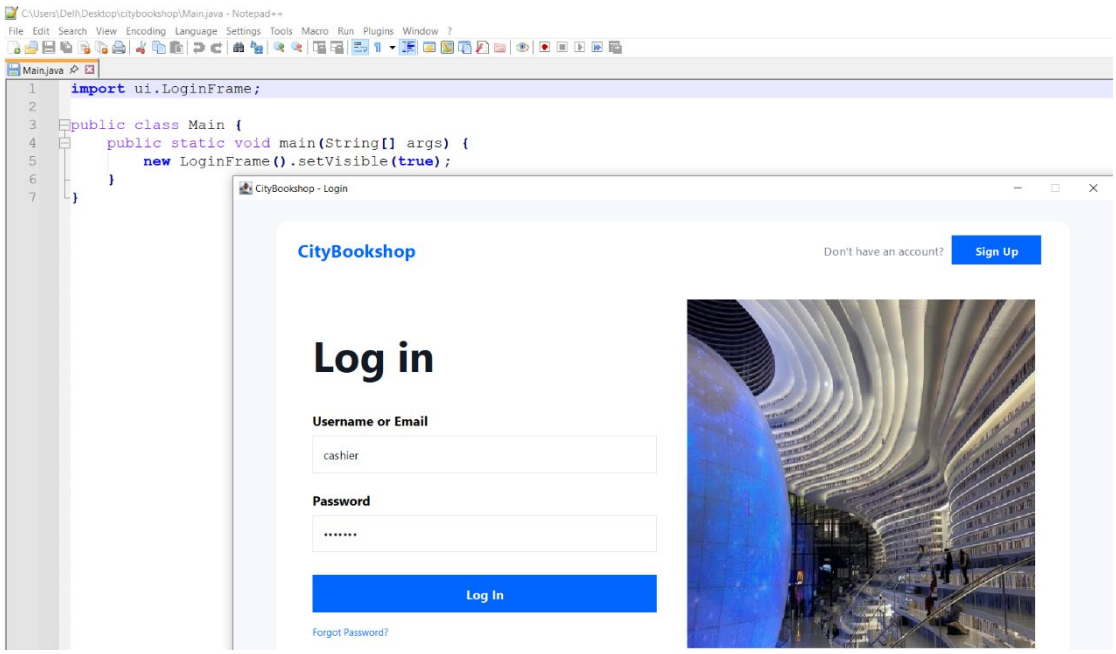


Figure 2.7.3

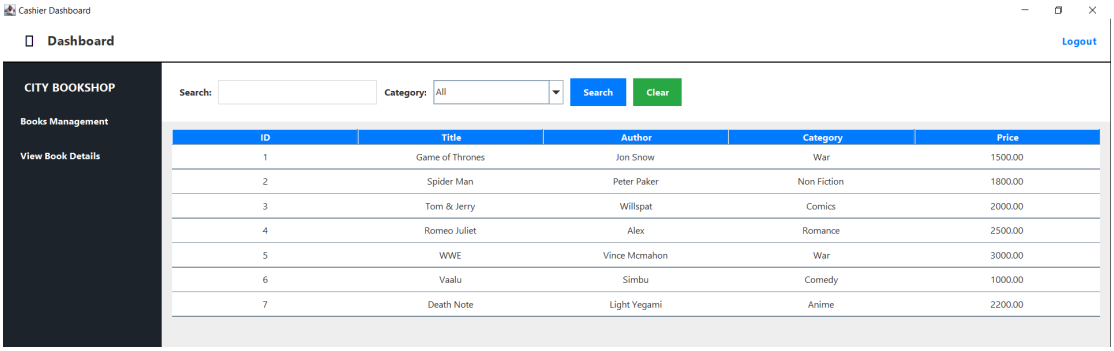


Figure 2.7.4

Test Case 3: Invalid Login

- **Input:** Username: wrong, Password: wrong
- **Expected Output:** Error message "Invalid username or password"
- **Result:** PASS

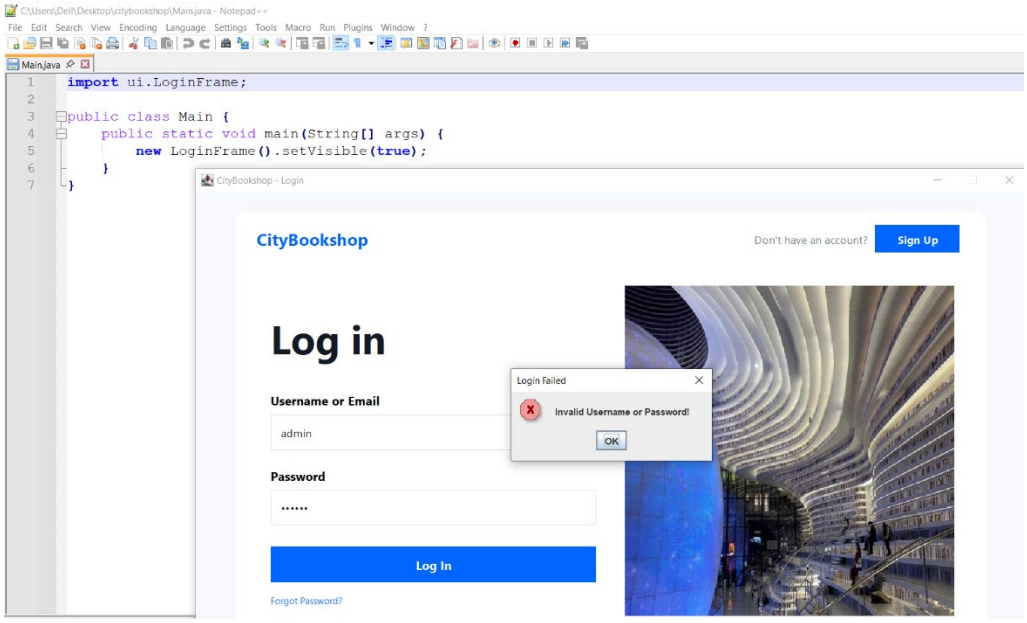


Figure 2.7.5

Test Case 4: View All Books

- **Input:** Click "View All Books" button
- **Expected Output:** Table displaying all books
- **Result:** PASS

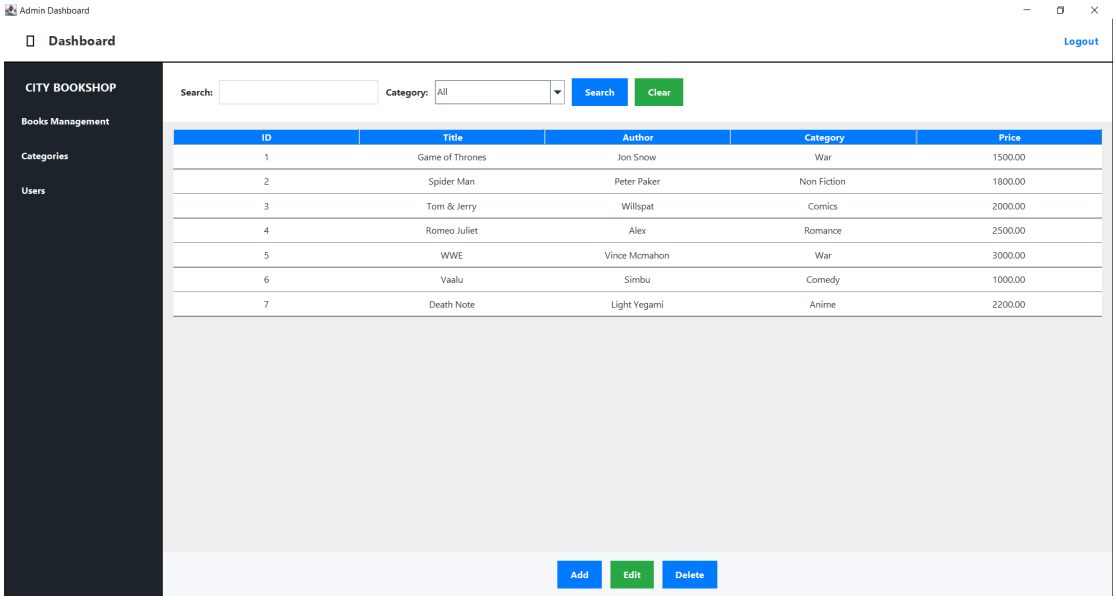


Figure 2.7.6

Test Case 5: Search Books by Title

- **Input:** Search type: Title, Search term: "Java"
- **Expected Output:** Books with "Java" in title displayed
- **Result:** PASS

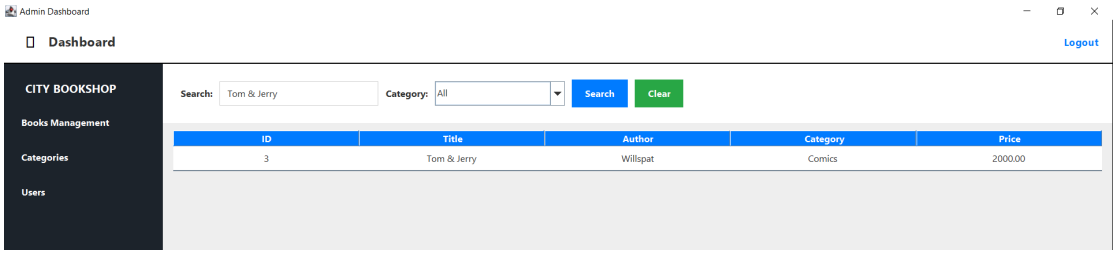


Figure 2.7.7

Test Case 6: Search Books by Category

- **Input:** Search type: Category, Search term: "Programming"
- **Expected Output:** Programming books displayed
- **Result:** PASS

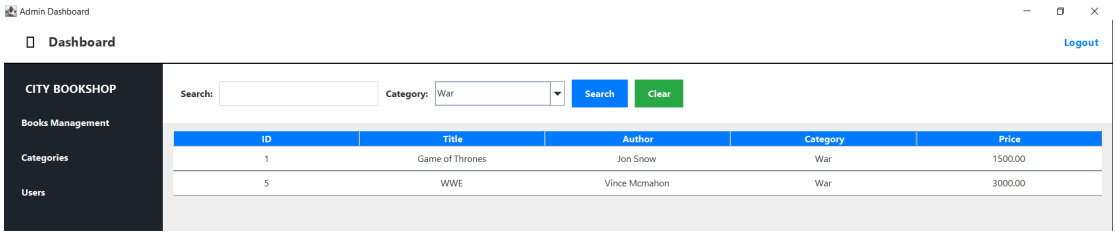


Figure 2.7.8

Test Case 7: Search Books by Price Range

- **Input:** Min Price: 10, Max Price: 30
- **Expected Output:** Books within price range displayed
- **Result:** PASS

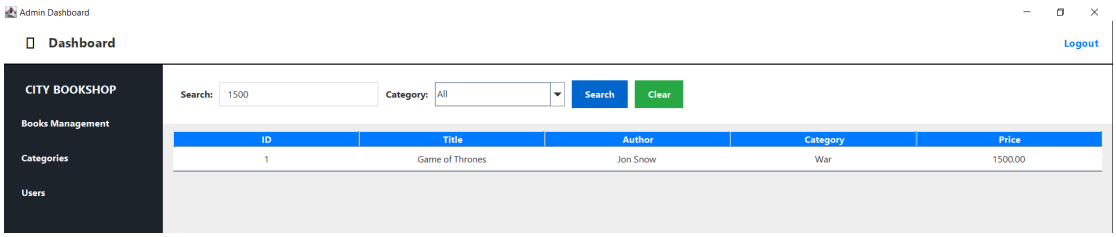


Figure 2.7.9

Test Case 8: Add New Book (Manager Only)

- **Input:** Title: "Test Book", Author: "Test Author", Category: "Programming", Price: 29.99, Stock: 10
- **Expected Output:** Book added successfully message
- **Result:** PASS

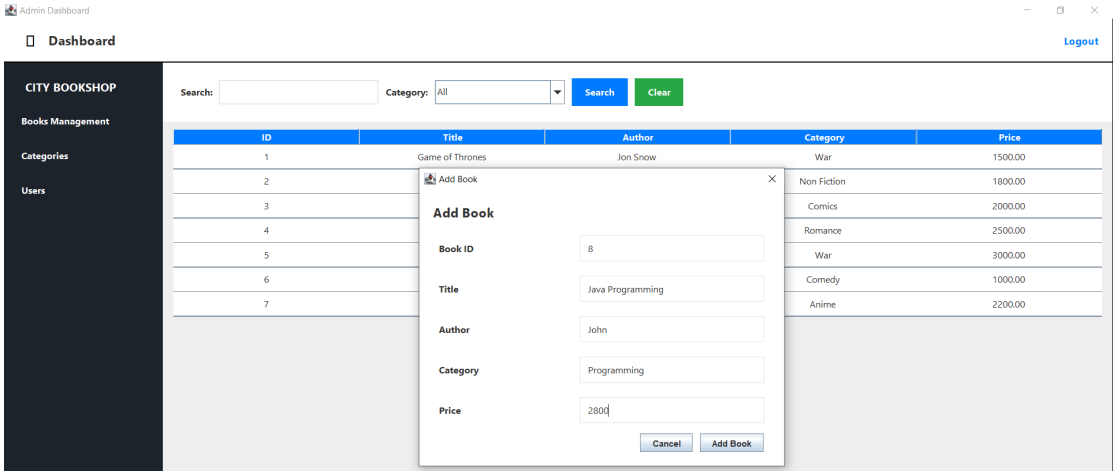


Figure 2.7.10

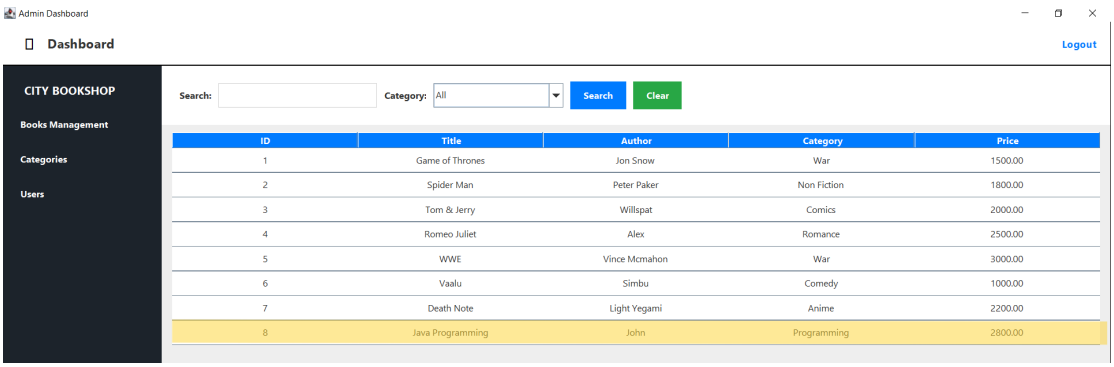


Figure 2.7.11

Test Case 9: Add New Category (Manager Only)

- **Input:** Name: “History”, Description: “History books”
- **Expected Output:** Category added successfully
- **Result:** PASS

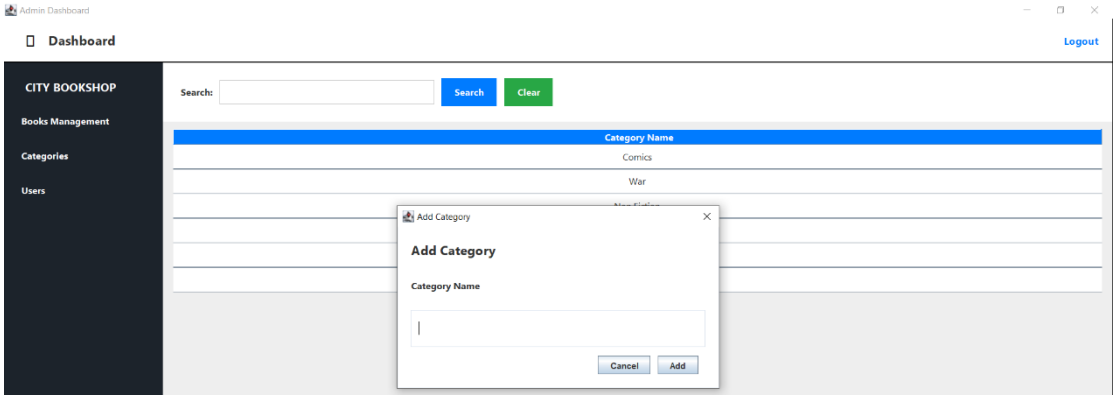


Figure 2.7.12

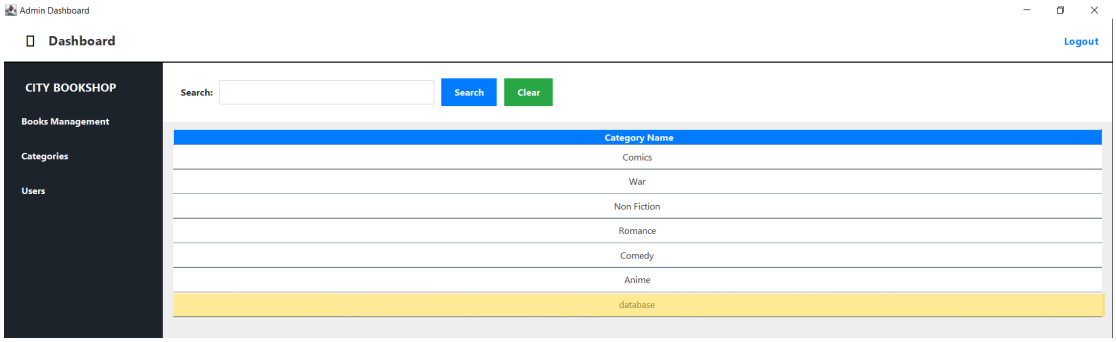


Figure 2.7.13

Test Case 10: Create New Account (Manager Only)

- **Input:** Full Name: “New User”, Username: “newuser”, Password: “pass123”, Type: “Cashier”
- **Expected Output:** Account created with new User ID
- **Result:** PASS

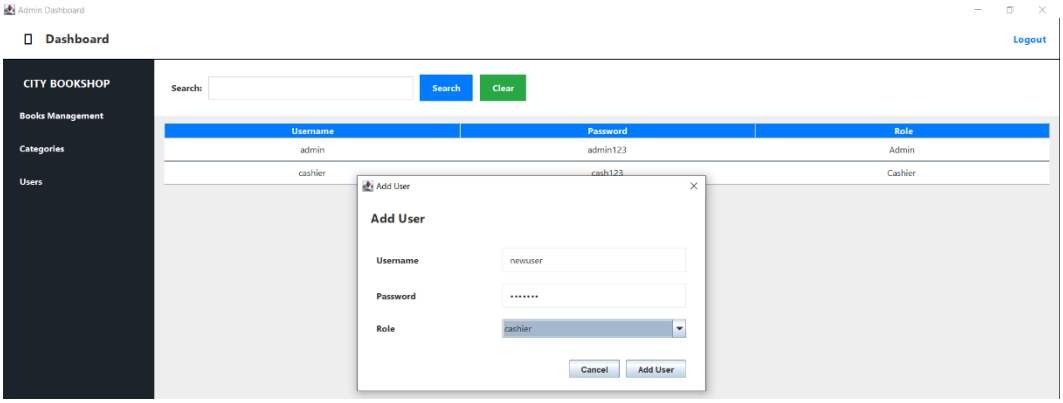


Figure 2.7.14

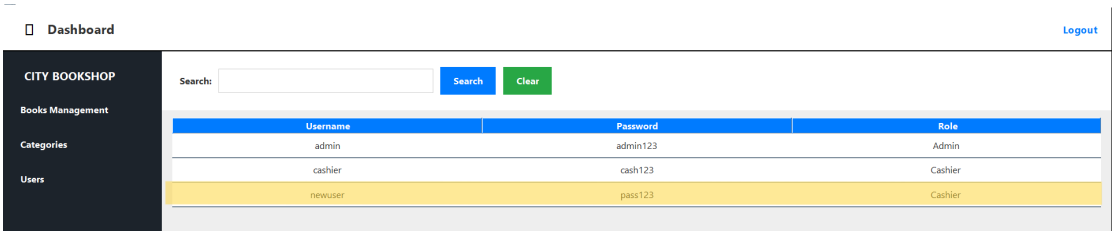


Figure 2.7.15

Chapter 3: Task 2 - User Manual

3.1 System Requirements

Minimum Hardware Requirements:

- Processor: 1 GHz or faster
- RAM: 512 MB minimum
- Storage: 50 MB free space
- Display: 1024x768 resolution

Software Requirements:

- Operating System: Windows 7/8/10/11, macOS, or Linux
- Java Development Kit (JDK) 8 or higher
- Java Runtime Environment (JRE) 8 or higher

3.2 Installation Guide

Step 1: Install Java

1. Download JDK from https://www.oracle.com/java/technologies/downloads/
2. Run the installer
3. Follow installation wizard
4. Verify installation by opening Command Prompt and typing: java -version

Step 2: Download Project Files

1. Copy all project files to a folder (e.g., C:\CityBookshop)

Step 3: Compile the Project

1. Open Command Prompt
2. Navigate to project folder: cd C:\CityBookshop
3. Compile: javac model/*.java service/*.java ui/*.java Main.java

Step 4: Run the Application

1. Execute: java Main
2. Login window will appear

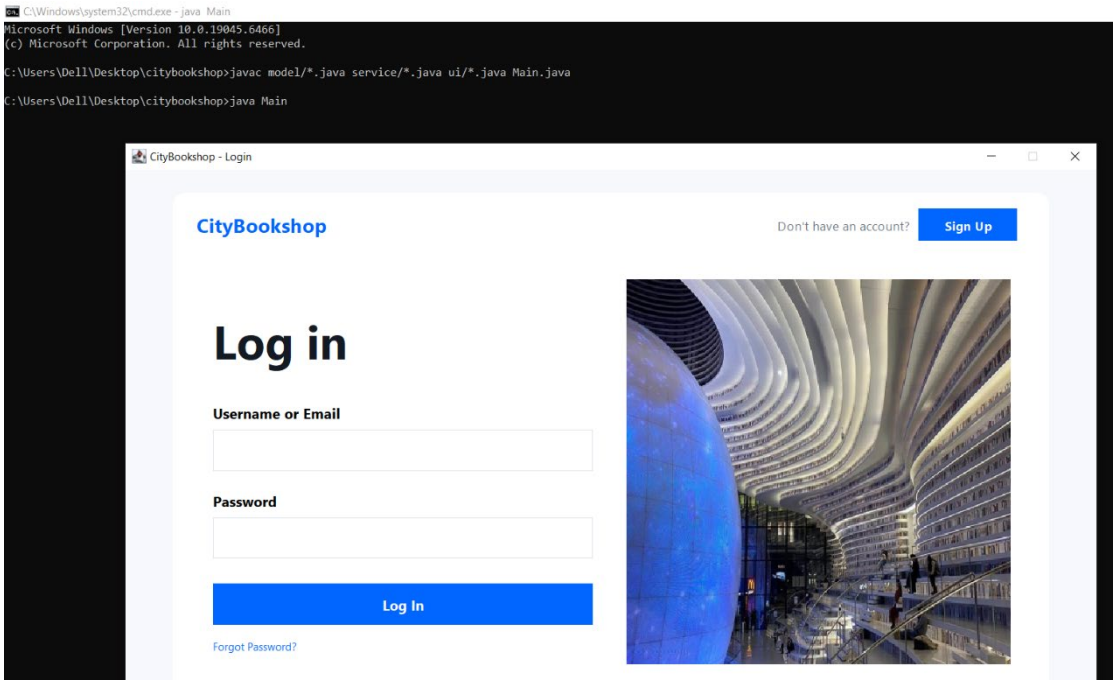


Figure 3.2.1

3.3 How to Use the System

3.3.1 Logging In

1. Enter your username
2. Enter your password
3. Click “Login” button
4. Default accounts:
    - Manager: admin / admin123
    - Cashier: cashier / cash123

3.3.2 Viewing All Books

1. Click “View All Books” button in the menu
2. All books will be displayed in a table
3. Table shows: Book ID, Title, Author, Category, Price, Stock

3.3.3 Searching for Books

1. Click “Search Books” button
2. Select search type from dropdown:
    - Title: Search by book title
    - Category: Search by category name
    - Price Range: Search by minimum and maximum price
3. Enter search criteria
4. Click “Search” button
5. Results will be displayed in the table

3.3.4 Adding a New Book (Manager Only)

1. Click “Add New Book” button
2. Fill in the form:
    - Title: Enter book title
    - Author: Enter author name
    - Category: Select from dropdown

- Price: Enter price (numbers only)
- Stock: Enter quantity (numbers only)

3. Click “Add Book” button
4. Success message will appear

3.3.5 Adding a New Category (Manager Only)

1. Click “Add Category” button
2. Enter category name
3. Enter description
4. Click “Add Category” button
5. New category will appear in the list

3.3.6 Creating a New Account (Manager Only)

1. Click “Create Account” button
2. Fill in the form:
   - Full Name: Enter user’s full name
   - Username: Enter login username
   - Password: Enter login password
   - Type: Select “Cashier” or “Manager”
3. Click “Create Account” button
4. User ID will be displayed in success message

3.3.7 Logging Out

1. Click “Logout” button
2. Confirm logout in the dialog
3. You will return to the login screen

3.4 Screenshots

	Screenshot Description	Figure No
1	Project Structure	Figure 2.6.1
2	Login Test (Manager)	Figure 2.7.1, Figure 2.7.2
3	Login Test (Cashier)	Figure 2.7.3, Figure 2.7.4
4	Invalid Login Test	Figure 2.7.5
5	View All Books Test	Figure 2.7.6
6	Search by Title Test	Figure 2.7.7
7	Search by Category Test	Figure 2.7.8
8	Search by Price Range Test	Figure 2.7.9
9	Add new Book Test	Figure 2.7.10, Figure 2.7.11
10	Add new Category Test	Figure 2.7.12
11	Add new User Test	Figure 2.7.13, Figure 2.7.14, Figure 2.7.15
13	Execute Commands	Figure 3.2.1

3.5 Troubleshooting

Problem	Solution
“Java is not recognized” error	Install JDK and add to PATH
Login fails with correct password	Check caps lock, re-type password
No books displayed	Ensure data folder exists with books.txt
Application won’t start	Recompile all files, check for errors
Price field shows error	Enter numbers only (no $ symbol)
Stock field shows error	Enter whole numbers only

Chapter 4: Conclusion

The City Bookshop Management System successfully demonstrates the application of Object-Oriented Programming concepts in a real-world scenario. The system fulfills all the requirements specified:

Achievements:

1. Implemented user authentication with two access levels (Manager, Cashier)

2. Cashier can view and search books by title, category, and price

3. Manager can add books, categories, and create new accounts

4. Data is saved and retrieved from text files

5. GUI implemented using Java Swing

OOP Concepts Demonstrated:

1. **Class & Object** - All entities defined as classes, instantiated as objects

2. **Encapsulation** - Private fields with public getters/setters

3. **Inheritance** - Manager and Cashier extend User class

4. **Abstraction** - User is an abstract class with abstract methods

5. **Polymorphism** - Different implementations of getUserType() and getMenuOptions()

Future Improvements:

- Add database support (MySQL/SQLite)

- Implement book sales/billing feature

- Add inventory tracking

- Generate sales reports

Chapter 5: References

1. Oracle Java Documentation - https://docs.oracle.com/javase/
2. Java Swing Tutorial - https://docs.oracle.com/javase/tutorial/uiswing/
3. Object-Oriented Programming Concepts - Course lecture notes
4. Java File I/O - https://docs.oracle.com/javase/tutorial/essential/io/

Quick Commands Reference

```
# Navigate to project
cd "c:\Users\fate\Desktop\BookShop"

# Compile all files
javac model/*.java service/*.java ui/*.java Main.java

# Run application
java Main

# Default logins:
# Manager: admin / admin123
# Cashier: cashier / cash123
```